

Curso: Introducción a Python

Python es un lenguaje de programación de alto nivel que se utiliza para crear programas informáticos de manera sencilla y clara.

Definición simple:

Python es un lenguaje que permite escribir instrucciones que la computadora puede entender, con una sintaxis fácil de leer y aprender.

Características principales:

- Fácil de aprender: su escritura es simple y parecida al lenguaje humano.
- Multiplataforma: funciona en Windows, Linux y Mac.
- Interpretado: no necesita compilarse antes de ejecutarse.
- Versátil: se usa en muchas áreas diferentes.

¿Para qué se usa?

Con Python se pueden hacer muchas cosas, por ejemplo:

- Programas de escritorio
- Páginas web
- Automatización de tareas
- Análisis de datos
- Inteligencia artificial

Módulo 1: Variables

¿Qué es una variable?

Una variable es un espacio en memoria donde almacenamos un valor.

Sintaxis básica:

```
</> Python  
  
nombre = "Juan"  
edad = 25
```

Reglas importantes:

- No usar espacios → ❌ `mi variable`
- No empezar con números → ❌ `1edad`
- Usar nombres descriptivos → ✅ `edad_usuario`

Ejemplo:

```
nombre = "Ana"
print(nombre)
```

Tipos de datos

Principales tipos en Python:

Tipo	Descripción	Ejemplo
<code>str</code>	Texto	<code>"Hola"</code>
<code>int</code>	Número entero	<code>10</code>
<code>float</code>	Número decimal	<code>3.14</code>
<code>bool</code>	Verdadero/Falso	<code>True</code> / <code>False</code>

Ejemplo:

```
nombre = "Carlos" # str
edad = 30          # int
altura = 1.75      # float
es_mayor = True    # bool
```

Ver tipo de dato:

```
print(type(nombre))
```

Salida de datos: `print()`

📌 ¿Para qué sirve?

Muestra información en pantalla.

🖋️ Ejemplos:

```
nombre = "Lucía"  
print("Hola", nombre)
```

💡 Concatenación:

```
print("Tu nombre es " + nombre)
```

Entrada de datos: **input()**

📌 ¿Para qué sirve?

Permite al usuario ingresar datos.

🔧 Ejemplo:

```
nombre = input("Ingresá tu nombre: ")  
print("Hola", nombre)
```

⚠️ Importante:

Todo lo que ingresa el usuario es **texto (string)** por defecto.

Conversiones de datos

📌 ¿Por qué son necesarias?

Porque **input()** devuelve siempre **str**.

🔄 Convertir a entero:

```
edad = int(input("Ingresá tu edad: "))
```

🔄 Convertir a decimal:

```
altura = float(input("Ingresá tu altura: "))
```

🔄 Convertir a texto:

```
numero = 10
texto = str(numero)
```

✓ Solución:

```
edad = int(input("Edad: "))
print(edad + 5)
```

⚠ Error común:

```
edad = input("Edad: ")
print(edad + 5) # ✗ Error
```

Ejercicio práctico

👉 Pedir al usuario que ingrese por teclado los siguientes datos:

- Nombre
- Edad
- Altura

Y mostrar por pantalla el resultado:

```
nombre = input("Nombre: ")
edad = int(input("Edad: "))
altura = float(input("Altura: "))

print("Hola", nombre)
print("Tenés", edad, "años")
print("Medís", altura, "metros")
```

Buenas prácticas

- Usar nombres claros en variables
- Evitar abreviaciones confusas
- Comentar el código cuando sea necesario
- Validar datos de entrada (nivel más avanzado)

2 Módulo 2: Operadores y Estructuras de Control

Operadores en Python

1 2 3 4 Operadores aritméticos

Operador	Descripción	Ejemplo
+	Suma	<code>5 + 2 = 7</code>
-	Resta	<code>5 - 2 = 3</code>
*	Multipliación	<code>5 * 2 = 10</code>
/	División	<code>5 / 2 = 2.5</code>
//	División entera	<code>5 // 2 = 2</code>
%	Módulo (resto)	<code>5 % 2 = 1</code>
**	Potencia	<code>2 ** 3 = 8</code>

 Ejemplo:

```
a = 10
b = 3

print(a + b)
print(a % b)
```

Operadores relacionales

Sirven para comparar valores → devuelven `True` o `False`

Operador	Significado
<code>==</code>	Igual
<code>!=</code>	Distinto
<code>></code>	Mayor
<code><</code>	Menor
<code>>=</code>	Mayor o igual
<code><=</code>	Menor o igual

 Ejemplo:

```
edad = 18
print(edad >= 18) # True
```

 Operadores lógicos

Operador	Uso
<code>and</code>	Ambas condiciones
<code>or</code>	Al menos una
<code>not</code>	Negación

 Ejemplo:

```
edad = 20
tiene_dni = True

print(edad >= 18 and tiene_dni)
```

Estructuras condicionales (if)

Sintaxis básica

```
if condicion:  
    # código
```

Condición con else

```
edad = 17  
  
if edad >= 18:  
    print("Mayor de edad")  
else:  
    print("Menor de edad")
```

Condiciones múltiples elif

```
nota = 7  
  
if nota >= 9:  
    print("Excelente")  
elif nota >= 6:  
    print("Aprobado")  
else:  
    print("Desaprobado")
```

Bucles (Loops)

Bucle while

 Repite mientras la condición sea verdadera

```
contador = 1

while contador <= 5:
    print(contador)
    contador += 1
```

⚠ Error común (loop infinito)

```
while True:
    print("Hola") # ⚠ nunca termina
```

Bucle for

 Se usa para recorrer secuencias

```
for i in range(5):
    print(i)
```

 range()

Uso	Resultado
<code>range(5)</code>	0 a 4
<code>range(1, 6)</code>	1 a 5
<code>range(1, 10, 2)</code>	de 2 en 2

 Ejemplo:

```
for i in range(1, 6):
    print("Número:", i)
```

Control de bucles

break (rompe el loop)

```
for i in range(10):  
    if i == 5:  
        break  
    print(i)
```

 continue (salta iteración)

```
for i in range(5):  
    if i == 2:  
        continue  
    print(i)
```

Ejercicios prácticos (nivel progresivo)

Nivel básico

Ejercicio 1:

Pedir un número y decir si es par o impar.

```
numero = int(input("Número: "))  
  
if numero % 2 == 0:  
    print("Par")  
else:  
    print("Impar")
```

Nivel intermedio

Ejercicio 2:

Mostrar los números del 1 al 10 con `for`.

Ejercicio 3:

Pedir números hasta que el usuario ingrese 0.

Nivel avanzado

Ejercicio 4:

Simular un login:

- Usuario correcto: `admin str, lower()`
 - Contraseña: `1234 int`
 - Máximo 3 intentos
-

Ejercicio integrador

Consigna:

Crear un programa que:

1. Pida edad
2. Pida si tiene registro (True/False)
3. Determine si puede conducir

Solución esperada:

```
edad = int(input("Edad: "))
registro = input("¿Tiene registro? (si/no): ")

if edad >= 18 and registro == "si":
    print("Puede conducir")
else:
    print("No puede conducir")
```

Buenas prácticas profesionales

- Validar inputs del usuario
- Evitar loops infinitos

- Usar nombres claros (`contador`, `edad_usuario`)
- Separar lógica en pasos simples
- Pensar en casos borde (QA mindset 🧐)

Módulo 3: Estructuras de Datos

Objetivo del módulo

Que el alumno pueda:

- Manejar colecciones de datos
 - Iterar estructuras
 - Aplicar lógica sobre listas y diccionarios
 - Detectar errores comunes (modo QA 🧐)
-

Listas (**list**)

¿Qué es una lista?

Una colección ordenada y mutable de elementos.

```
numeros = [1, 2, 3, 4]
nombres = ["Ana", "Luis", "Carlos"]
```

Operaciones básicas

Acceder a elementos

```
print(nombres[0]) # Ana
print(nombres[-1]) # Carlos
```

Modificar

```
nombres[1] = "Pedro"
```

Agregar

```
nombres.append("María")
```

Eliminar

```
nombres.remove("Ana")
```

Longitud

```
print(len(nombres))
```

Recorrer listas

```
for nombre in nombres:  
    print(nombre)
```

Errores comunes (QA mindset)

- Índices fuera de rango → `nombres[10]`
- Eliminar elemento inexistente → `remove("X")`
- Lista vacía

Tuplas (**tuple**)

Característica principal

Son **inmutables** (no se pueden modificar)

```
coordenadas = (10, 20)
```

Uso típico

- Datos que no cambian (ej: posiciones, configs)
-

Error común

```
coordenadas[0] = 5 # ✗ rompe
```

Diccionarios (**dict**)

¿Qué es?

Estructura clave → valor

```
usuario = {  
    "nombre": "Juan",  
    "edad": 30  
}
```

Operaciones

Acceder

```
print(usuario["nombre"])
```

Modificar

```
usuario["edad"] = 31
```

Agregar

```
usuario["email"] = "juan@mail.com"
```

Recorrer

```
for clave, valor in usuario.items():  
    print(clave, valor)
```

Listas de diccionarios (🔥 MUY IMPORTANTE)

👉 Esto es lo más usado en sistemas reales (APIs, datos backend, testing)

```
usuarios = [  
    {"nombre": "Ana", "edad": 25},  
    {"nombre": "Luis", "edad": 30}  
]
```

🔄 Recorrido

```
for usuario in usuarios:  
    print(usuario["nombre"])
```

🧪 Ejemplo real tipo QA

Filtrar usuarios mayores de edad:

```
for usuario in usuarios:  
    if usuario["edad"] >= 18:  
        print(usuario["nombre"])
```

Ejercicios prácticos

● Básico

Crear una lista de números y mostrar el mayor.

Intermedio

Guardar 3 usuarios en lista de diccionarios y mostrar sus nombres.

Avanzado

Simular un sistema que:

- Guarda usuarios
 - Permite buscar por nombre
 - Valida si existe
-

Evaluación tipo QA (🔥 clave para vos)

Objetivo

Detectar errores, casos borde y comportamientos inesperados.

Caso 1: Lista vacía

```
numeros = []  
print(numeros[0])
```

- 👉 ? ¿Qué pasa?
- 👉 ? ¿Cómo lo validarías?

✓ Esperado QA:

- Error: **IndexError**
 - Validar longitud antes de acceder
-

Caso 2: Eliminación inválida

```
nombres = ["Ana", "Luis"]  
nombres.remove("Pedro")
```

👉 ? ¿Qué sucede?

✓ Esperado:

- **ValueError**
 - Validar existencia antes
-

Caso 3: Diccionario incompleto

```
usuario = {"nombre": "Ana"}  
print(usuario["edad"])
```

👉 ? Resultado?

✓ Esperado:

- **KeyError**
- Usar **.get()**

```
usuario.get("edad", "No definido")
```

Caso 4: Tipos incorrectos

```
edad = "20"  
  
if edad > 18:  
    print("Mayor")
```


👉 ? Problema?

✓ Esperado:

- Comparación inválida
 - Convertir a int
-

🧩 Caso 5: Datos inconsistentes (REAL QA)

```
usuarios = [  
    {"nombre": "Ana", "edad": 25},  
    {"nombre": "Luis"},  
]
```

```
for u in usuarios:  
    print(u["edad"])
```

👉 ? Qué validarías?

✓ QA mindset:

- Datos incompletos
 - Validar estructura antes de usar
-

🧩 Caso 6: Input inesperado

```
edad = int(input("Edad: "))
```

👉 Usuario ingresa: "abc"

✓ Resultado:

- **ValueError**

✓ Solución QA:

```
try:
    edad = int(input("Edad: "))
except:
    print("Dato inválido")
```

🧩 Caso 7: Bucle con datos corruptos

```
usuarios = None

for u in usuarios:
    print(u)
```

👉 ? Qué pasa?

✓ Error:

- **TypeError**

✓ Validación:

```
if usuarios:
    ...
```

🧠 Enfoque QA PRO (esto te diferencia)

Cuando veas código pensá:

- ❌ ¿Qué pasa si viene vacío?
- ❌ ¿Qué pasa si viene null?
- ❌ ¿Qué pasa si el tipo es incorrecto?

- ❌ ¿Qué pasa si falta un campo?
 - ❌ ¿Qué pasa si hay más datos de lo esperado?
-

Ejercicio tipo entrevista QA + Python

Consigna

Dado:

```
usuarios = [  
    {"nombre": "Ana", "edad": 25},  
    {"nombre": "Luis", "edad": "30"},  
    {"nombre": "Carlos"},  
    {},  
]
```

👉 Crear un programa que:

- Muestre solo usuarios válidos
 - Edad debe ser número
 - Debe existir nombre y edad
-

✓ Solución esperada:

```
for u in usuarios:
    if "nombre" in u and "edad" in u:
        try:
            edad = int(u["edad"])
            print(u["nombre"], edad)
        except:
            print("Edad inválida en:", u)
    else:
        print("Usuario incompleto:", u)
```

4 Módulo 4: Funciones, manejo de errores y archivos

1. Funciones

¿Qué es una función?

Una función es un bloque de código reutilizable que ejecuta una tarea específica.

Sirve para:

- **organizar mejor el código**
- **evitar repeticiones**
- **hacer programas más claros**
- **facilitar pruebas y mantenimiento**

Sintaxis básica

```
def saludar():
    print("Hola")
```

Ejemplo de uso

```
def saludar():  
    print("Hola")  
  
saludar()
```

Funciones con parámetros

Los parámetros permiten que la función reciba datos.

```
def saludar(nombre):  
    print("Hola", nombre)  
  
saludar("Ana")  
saludar("Luis")
```

Funciones con varios parámetros

```
def sumar(a, b):  
    print(a + b)  
  
sumar(3, 5)
```

Funciones que retornan valores

Cuando necesitamos que la función nos devuelva un resultado, usamos **return**.

```
def sumar(a, b):  
    return a + b  
  
resultado = sumar(10, 20)  
print(resultado)
```

Diferencia entre **print()** y **return**

```
def con_print():  
    print(5)  
  
def con_return():  
    return 5
```

- **print()** muestra un valor
- **return** lo devuelve para seguir trabajándolo

Ejemplo:

```
def multiplicar(a, b):  
    return a * b  
  
resultado = multiplicar(4, 3)  
print("El resultado es:", resultado)
```

Buenas prácticas con funciones

- usar nombres claros
- que cada función haga una sola cosa
- evitar funciones demasiado largas
- reutilizar lógica repetida

Ejemplo correcto:

```
def es_mayor_de_edad(edad):  
    return edad >= 18
```

2. Manejo de errores

¿Qué es un error?

Un error ocurre cuando el programa encuentra una situación que no puede resolver correctamente.

Ejemplo:

```
edad = int(input("Ingresá tu edad: "))
```

Si el usuario escribe **hola**, el programa falla.

try y except

Permiten controlar errores sin que el programa se rompa.

```
try:
    edad = int(input("Ingresá tu edad: "))
    print("Tu edad es:", edad)
except:
    print("Ingresaste un valor inválido")
```

Ejemplo más real

```
try:
    numero1 = int(input("Primer número: "))
    numero2 = int(input("Segundo número: "))
    print(numero1 / numero2)
except:
    print("Ocurrió un error")
```

Manejo específico de errores

Es mejor capturar errores concretos.

```
try:
    numero1 = int(input("Primer número: "))
    numero2 = int(input("Segundo número: "))
    print(numero1 / numero2)
except ValueError:
    print("Debés ingresar números válidos")
except ZeroDivisionError:
    print("No se puede dividir por cero")
```

else y finally

```
try:
    numero = int(input("Número: "))
except ValueError:
    print("Dato inválido")
else:
    print("Todo salió bien")
finally:
    print("Fin del proceso")
```

Significado

- **try:** código a intentar
 - **except:** qué hacer si falla
 - **else:** qué hacer si no falla
 - **finally:** se ejecuta siempre
-

3. Archivos

¿Para qué sirven?

Permiten guardar información fuera del programa.

Ejemplos reales:

- guardar usuarios
 - registrar operaciones
 - exportar resultados
 - leer configuración
-

Abrir un archivo

```
archivo = open("datos.txt", "r")
```

Pero la mejor práctica es usar **with**.

Leer un archivo

```
with open("datos.txt", "r", encoding="utf-8") as archivo:  
    contenido = archivo.read()  
    print(contenido)
```

Escribir un archivo

```
with open("datos.txt", "w", encoding="utf-8") as archivo:  
    archivo.write("Hola mundo")
```

Importante

- "w" sobrescribe el contenido existente
 - si el archivo no existe, lo crea
-

Agregar contenido a un archivo

```
with open("datos.txt", "a", encoding="utf-8") as archivo:  
    archivo.write("\nNueva línea")
```

Leer línea por línea

```
with open("datos.txt", "r", encoding="utf-8") as archivo:  
    for linea in archivo:  
        print(linea.strip())
```

Ejemplo práctico

```
nombre = input("Ingresá tu nombre: ")  
  
with open("usuarios.txt", "a", encoding="utf-8") as archivo:  
    archivo.write(nombre + "\n")  
  
print("Usuario guardado correctamente")
```

4. Integración de conceptos

Ejemplo completo

```
def guardar_usuario(nombre, edad):  
    with open("usuarios.txt", "a", encoding="utf-8") as archivo:  
        archivo.write(f"{nombre},{edad}\n")  
  
try:  
    nombre = input("Nombre: ")  
    edad = int(input("Edad: "))  
    guardar_usuario(nombre, edad)  
    print("Usuario guardado")  
except ValueError:  
    print("La edad debe ser numérica")
```

Acá se combinan:

- funciones
 - input
 - conversiones
 - manejo de errores
 - archivos
-

5. Ejercicios prácticos

Ejercicio 1

Crear una función que reciba un nombre y lo salude.

Ejercicio 2

Crear una función que reciba dos números y devuelva el mayor.

Ejercicio 3

Pedir al usuario dos números y mostrar el resultado de la división, controlando errores.

Ejercicio 4

Crear un archivo llamado `notas.txt` y guardar tres notas ingresadas por el usuario.

Ejercicio 5

Leer el archivo **notas.txt** y mostrar su contenido línea por línea.

6. Evaluación tipo QA del módulo 4

Caso borde 1

```
def dividir(a, b):  
    return a / b  
  
print(dividir(10, 0))
```

Qué validar

- división por cero
 - mensaje de error claro
 - que el programa no se caiga abruptamente
 - Que permita salir del programa
-

Caso borde 2

```
def obtener_edad():  
    edad = int(input("Edad: "))  
    return edad
```

Qué validar

- input vacío
- letras
- caracteres especiales
- números negativos
- edad exageradamente alta

Caso borde 3

```
with open("archivo_que_no_existe.txt", "r", encoding="utf-8") as archivo:  
    print(archivo.read())
```

Qué validar

- archivo inexistente
- permisos insuficientes
- rutas mal escritas

Caso borde 4

```
def guardar(nombre):  
    with open("usuarios.txt", "a", encoding="utf-8") as archivo:  
        archivo.write(nombre)
```

Qué validar

- nombre vacío
- espacios solamente
- caracteres extraños
- múltiples registros sin salto de línea

7. Resumen del módulo

En este módulo aprendiste a:

- crear funciones
- usar parámetros y **return**

- manejar errores con **try** y **except**
 - leer y escribir archivos
 - combinar todo en programas más reales
-

TP Final tipo proyecto real: Simulación de backend

Nombre del proyecto

Sistema de gestión de usuarios

Objetivo

Construir un programa en Python que simule el comportamiento básico de un backend de usuarios.

El sistema deberá permitir:

- crear usuarios
 - listar usuarios
 - buscar usuarios por ID
 - eliminar usuarios
 - validar datos
 - guardar datos en archivo
-

Contexto funcional

Imaginemos que una aplicación necesita administrar usuarios registrados.

No vamos a usar base de datos real ni API real todavía, pero vamos a simular la lógica de backend con Python.

Cada usuario tendrá:

- **id**
 - **nombre**
 - **email**
 - **edad**
-

1. Requerimientos funcionales

RF01 - Alta de usuario

El sistema debe permitir crear un usuario con:

- **id numérico**
- **nombre no vacío**
- **email no vacío**
- **edad numérica mayor o igual a 0**

RF02 - Listado de usuarios

El sistema debe mostrar todos los usuarios guardados.

RF03 - Búsqueda por ID

El sistema debe permitir buscar un usuario por su **id.**

RF04 - Eliminación de usuario

El sistema debe permitir eliminar un usuario por su **id.**

RF05 - Persistencia

Los usuarios deben guardarse en archivo para no perder información al cerrar el programa.

2. Reglas de negocio

- no puede haber dos usuarios con el mismo **id**
 - el nombre no puede estar vacío
 - el email no puede estar vacío
 - la edad debe ser un número entero
 - la edad no puede ser negativa
-

3. Modelo de datos sugerido

Ejemplo de usuario:

```
{  
  "id": 1,  
  "nombre": "Ana",  
  "email": "ana@mail.com",  
  "edad": 25  
}
```

Se puede trabajar con una lista de diccionarios.

4. Implementación sugerida

Versión simple en memoria

```
usuarios = []
```

Funciones sugeridas


```
def crear_usuario(usuarios, id_usuario, nombre, email, edad):
    pass

def listar_usuarios(usuarios):
    pass

def buscar_usuario_por_id(usuarios, id_usuario):
    pass

def eliminar_usuario(usuarios, id_usuario):
    pass
```

5. Solución ejemplo

```
def cargar_usuarios():
    if not os.path.exists(ARCHIVO_USUARIOS):
        return []

    try:
        with open(ARCHIVO_USUARIOS, "r", encoding="utf-8") as archivo:
            return json.load(archivo)
    except (json.JSONDecodeError, OSError):
        return []

def guardar_usuarios(usuarios):
    with open(ARCHIVO_USUARIOS, "w", encoding="utf-8") as archivo:
        json.dump(usuarios, archivo, ensure_ascii=False, indent=4)
```

```
def validar_usuario(id_usuario, nombre, email, edad):
    if not isinstance(id_usuario, int):
        return False, "El id debe ser numérico"

    if not nombre or not nombre.strip():
        return False, "El nombre es obligatorio"

    if not email or not email.strip():
        return False, "El email es obligatorio"

    if not isinstance(edad, int):
        return False, "La edad debe ser numérica"

    if edad < 0:
        return False, "La edad no puede ser negativa"

    return True, "Usuario válido"
```

```
def existe_id(usuarios, id_usuario):
    for usuario in usuarios:
        if usuario["id"] == id_usuario:
            return True
    return False

def crear_usuario(usuarios, id_usuario, nombre, email, edad):
    valido, mensaje = validar_usuario(id_usuario, nombre, email, edad)

    if not valido:
        return False, mensaje

    if existe_id(usuarios, id_usuario):
        return False, "Ya existe un usuario con ese id"
```

```
    usuario = {
        "id": id_usuario,
        "nombre": nombre.strip(),
        "email": email.strip(),
        "edad": edad
    }

    usuarios.append(usuario)
    guardar_usuarios(usuarios)
    return True, "Usuario creado correctamente"

def listar_usuarios(usuarios):
    return usuarios
```

```
def buscar_usuario_por_id(usuarios, id_usuario):
    for usuario in usuarios:
        if usuario["id"] == id_usuario:
            return usuario
    return None

def eliminar_usuario(usuarios, id_usuario):
    usuario = buscar_usuario_por_id(usuarios, id_usuario)

    if usuario is None:
        return False, "Usuario no encontrado"

    usuarios.remove(usuario)
    guardar_usuarios(usuarios)
    return True, "Usuario eliminado correctamente"
```

```

def mostrar_menu():
    print("\n--- Sistema de usuarios ---")
    print("1. Crear usuario")
    print("2. Listar usuarios")
    print("3. Buscar usuario por ID")
    print("4. Eliminar usuario")
    print("5. Salir")

def main():
    usuarios = cargar_usuarios()

    while True:
        mostrar_menu()
        opcion = input("Elegí una opción: ")

```

```

if opcion == "1":
    try:
        id_usuario = int(input("ID: "))
        nombre = input("Nombre: ")
        email = input("Email: ")
        edad = int(input("Edad: "))
        ok, mensaje = crear_usuario(usuarios, id_usuario, nombre, email, edad)
        print(mensaje)
    except ValueError:
        print("ID y edad deben ser numéricos")

elif opcion == "2":
    lista = listar_usuarios(usuarios)
    if not lista:
        print("No hay usuarios cargados")
    else:
        for usuario in lista:
            print(usuario)

```

```

elif opcion == "3":
    try:
        id_usuario = int(input("Ingresá el ID a buscar: "))
        usuario = buscar_usuario_por_id(usuarios, id_usuario)
        if usuario:
            print(usuario)
        else:
            print("Usuario no encontrado")
    except ValueError:
        print("El ID debe ser numérico")

```

```
elif opcion == "5":  
    print("Fin del programa")  
    break  
  
else:  
    print("Opción inválida")  
  
if __name__ == "__main__":  
    main()
```



Casos de prueba manuales

CP01 - Crear usuario válido

Precondición: sistema iniciado.

Pasos:

1. Seleccionar opción crear usuario.
2. Ingresar ID válido.
3. Ingresar nombre válido.
4. Ingresar email válido.
5. Ingresar edad válida.

Resultado esperado:

El sistema crea el usuario y muestra mensaje de éxito.

CP02 - Crear usuario con ID duplicado

Precondición: existe un usuario con ID 1.

Pasos:

1. Intentar crear otro usuario con ID 1.
2. Completar los demás campos correctamente.

Resultado esperado:

El sistema rechaza la operación e informa que el ID ya existe.

CP03 - Crear usuario con nombre vacío

Pasos:

- 1. Ingresar ID válido.**
- 2. Dejar nombre vacío.**
- 3. Completar el resto.**

Resultado esperado:

El sistema no crea el usuario y muestra validación de nombre obligatorio.

CP04 - Crear usuario con email vacío

Resultado esperado:

El sistema rechaza el alta e informa que el email es obligatorio.

CP05 - Crear usuario con edad negativa

Resultado esperado:

El sistema rechaza el alta e informa que la edad no puede ser negativa.

CP06 - Buscar usuario existente

Precondición: existe usuario con ID 2.

Pasos:

- 1. Ir a búsqueda.**
- 2. Ingresar ID 2.**

Resultado esperado:

Se visualizan correctamente los datos del usuario.

CP07 - Buscar usuario inexistente

Resultado esperado:

El sistema informa que el usuario no fue encontrado.

CP08 - Eliminar usuario existente

Resultado esperado:

El sistema elimina el usuario y lo confirma.

CP09 - Eliminar usuario inexistente

Resultado esperado:

El sistema informa que no existe el usuario a eliminar.

CP10 - Persistencia de datos

Pasos:

- 1. Crear usuario válido.**
- 2. Cerrar programa.**

3. Volver a ejecutar.

4. Listar usuarios.

Resultado esperado:

El usuario creado sigue existiendo.

Casos borde y trampas de QA

Caso borde 1: ID como texto

Entrada:

```
"id" = "abc"
```

Esperado: validación de tipo.

Caso borde 2: Nombre con espacios

Entrada:

```
nombre = " "
```

Esperado: no se acepta.

Caso borde 3: Email con espacios

Entrada:

```
email = " "
```

Esperado: no se acepta.

Caso borde 4: Edad como texto no numérico

Entrada:

```
edad = "veinte"
```

Esperado: error controlado.

Caso borde 5: Edad decimal

Entrada:

```
edad = 20.5
```

Esperado: el sistema debería rechazarlo si la regla exige entero.

Caso borde 6: Archivo corrupto

usuarios.json contiene texto inválido.

Esperado: el sistema no se rompe y carga lista vacía o informa el problema.

Caso borde 7: Eliminación doble

Eliminar el mismo usuario dos veces.

Esperado: la primera vez éxito, la segunda vez usuario no encontrado.

Caso borde 8: Búsqueda con input vacío

Esperado: validación de entrada.



Automatización con Python

Para automatizar pruebas vamos a separar la lógica del programa de la interacción por consola.

Eso permite testear funciones directamente.

La automatización más común en Python se hace con `pytest`.

1. Archivo del sistema

Guardá la lógica en un archivo llamado:

```
backend_usuarios.py
```

2. Casos automatizados con `pytest`

Crear archivo:

```
test_backend_usuarios.py
```

Contenido:

```
from backend_usuarios import (
    validar_usuario,
    crear_usuario,
    buscar_usuario_por_id,
    eliminar_usuario
)

def test_validar_usuario_ok():
    ok, mensaje = validar_usuario(1, "Ana", "ana@mail.com", 25)
    assert ok is True
    assert mensaje == "Usuario válido"

def test_validar_usuario_nombre_vacio():
    ok, mensaje = validar_usuario(1, "", "ana@mail.com", 25)
    assert ok is False
    assert mensaje == "El nombre es obligatorio"
```

```
def test_validar_usuario_email_vacio():
    ok, mensaje = validar_usuario(1, "Ana", "", 25)
    assert ok is False
    assert mensaje == "El email es obligatorio"

def test_validar_usuario_edad_negativa():
    ok, mensaje = validar_usuario(1, "Ana", "ana@mail.com", -1)
    assert ok is False
    assert mensaje == "La edad no puede ser negativa"

def test_crear_usuario_ok():
    usuarios = []
    ok, mensaje = crear_usuario(usuarios, 1, "Ana", "ana@mail.com", 25)
    assert ok is True
    assert mensaje == "Usuario creado correctamente"
    assert len(usuarios) == 1
    assert usuarios[0]["nombre"] == "Ana"
```

```
def test_crear_usuario_id_duplicado():
    usuarios = [{"id": 1, "nombre": "Ana", "email": "ana@mail.com", "edad": 25}]
    ok, mensaje = crear_usuario(usuarios, 1, "Luis", "luis@mail.com", 30)
    assert ok is False
    assert mensaje == "Ya existe un usuario con ese id"
    assert len(usuarios) == 1

def test_buscar_usuario_existente():
    usuarios = [{"id": 1, "nombre": "Ana", "email": "ana@mail.com", "edad": 25}]
    usuario = buscar_usuario_por_id(usuarios, 1)
    assert usuario is not None
    assert usuario["nombre"] == "Ana"

def test_buscar_usuario_inexistente():
    usuarios = [{"id": 1, "nombre": "Ana", "email": "ana@mail.com", "edad": 25}]
    usuario = buscar_usuario_por_id(usuarios, 2)
    assert usuario is None
```

```
def test_eliminar_usuario_existente():
    usuarios = [{"id": 1, "nombre": "Ana", "email": "ana@mail.com", "edad": 25}]
    ok, mensaje = eliminar_usuario(usuarios, 1)
    assert ok is True
    assert mensaje == "Usuario eliminado correctamente"
    assert len(usuarios) == 0

def test_eliminar_usuario_inexistente():
    usuarios = [{"id": 1, "nombre": "Ana", "email": "ana@mail.com", "edad": 25}]
    ok, mensaje = eliminar_usuario(usuarios, 2)
    assert ok is False
    assert mensaje == "Usuario no encontrado"
```

3. Cómo ejecutar las pruebas

Instalar pytest

```
</> Bash
pip install pytest
```

Ejecutar tests

```
❏ Bash
```

```
pytest
```

Ejecutar con detalle

```
❏ Bash
```

```
pytest -v
```

4. Mejora importante para testing

En un proyecto real, conviene que las funciones que escriben archivos puedan testearse sin tocar el archivo real de producción.

Por ejemplo, una mejora sería:

```
def guardar_usuarios(usuarios, ruta_archivo="usuarios.json"):
    with open(ruta_archivo, "w", encoding="utf-8") as archivo:
        json.dump(usuarios, archivo, ensure_ascii=False, indent=4)
```

Así en tests se puede usar un archivo temporal.

5. Ejemplo de prueba con archivo temporal

```
import json
from backend_usuarios import guardar_usuarios

def test_guardar_usuarios(tmp_path):
    ruta = tmp_path / "usuarios_test.json"
    usuarios = [{"id": 1, "nombre": "Ana", "email": "ana@mail.com", "edad": 25}]

    guardar_usuarios(usuarios, ruta)

    with open(ruta, "r", encoding="utf-8") as archivo:
        data = json.load(archivo)

    assert len(data) == 1
    assert data[0]["email"] == "ana@mail.com"
```

5 Módulo 5: JSON, APIs y Testing orientado a Backend

1. Introducción a JSON

¿Qué es JSON?

JSON significa JavaScript Object Notation.

Es un formato de texto muy usado para intercambiar datos entre sistemas.

Se utiliza muchísimo en:

- **APIs**
 - **backends**
 - **frontends**
 - **configuraciones**
 - **exportación e importación de datos**
-

Ejemplo de JSON

```
</> JSON

{
  "id": 1,
  "nombre": "Ana",
  "email": "ana@mail.com",
  "activo": true
}
```

Equivalencia en Python

En Python, ese JSON se representa como un diccionario:

```
</> Python

usuario = {
    "id": 1,
    "nombre": "Ana",
    "email": "ana@mail.com",
    "activo": True
}
```

Tipos compatibles entre JSON y Python

JSON	Python
string	str
number	int / float
true / false	True / False
null	None
object	dict
array	list

2. Trabajar con JSON en Python

Python tiene un módulo incorporado llamado **json**.

```
import json
```

Convertir diccionario a JSON

```
import json

usuario = {
    "id": 1,
    "nombre": "Ana",
    "email": "ana@mail.com"
}

texto_json = json.dumps(usuario)
print(texto_json)
```

Qué hace **dumps()**

Convierte un objeto Python en texto JSON.

Convertir JSON a diccionario

```
import json

texto_json = '{"id": 1, "nombre": "Ana"}'
usuario = json.loads(texto_json)

print(usuario)
print(type(usuario))
```

Qué hace **loads()**

Convierte texto JSON en un objeto Python.

Guardar JSON en archivo

```
import json

usuario = {
    "id": 1,
    "nombre": "Ana",
    "email": "ana@mail.com"
}

with open("usuario.json", "w", encoding="utf-8") as archivo:
    json.dump(usuario, archivo, ensure_ascii=False, indent=4)
```

Leer JSON desde archivo

```
import json

with open("usuario.json", "r", encoding="utf-8") as archivo:
    datos = json.load(archivo)

print(datos)
```

3. ¿Qué es una API?

Definición simple

Una API permite que dos sistemas se comuniquen.

Ejemplo:

- una app web le pide datos al backend
- el backend responde con JSON

Ejemplo de la vida real

Cuando una app muestra una lista de usuarios, muchas veces está haciendo una llamada a una API como:


```
<> http  
GET /usuarios
```

Y el backend responde algo así:

```
<> JSON  
  
[  
  {  
    "id": 1,  
    "nombre": "Ana"  
  },  
  {  
    "id": 2,  
    "nombre": "Luis"  
  }  
]
```

4. Conceptos básicos de APIs REST

Métodos HTTP principales

Método	Uso
GET	Obtener datos
POST	Crear datos
PUT	Modificar todo un recurso
PATCH	Modificar parcialmente
DELETE	Eliminar

Ejemplos conceptuales

- **GET /usuarios** → listar usuarios

- **GET /usuarios/1** → obtener usuario con id 1
 - **POST /usuarios** → crear usuario
 - **PUT /usuarios/1** → actualizar usuario
 - **DELETE /usuarios/1** → eliminar usuario
-

5. Status codes más comunes

Código	Significado
200	OK
201	Created
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error

Ejemplo QA

Si hacés un **GET** a un usuario existente:

- esperado: **200**

Si buscás uno que no existe:

- esperado: **404**

Si enviás datos inválidos en un alta:

- esperado: **400**

6. Consumir APIs con Python

Para consumir APIs normalmente se usa la librería **requests**.

Instalación

```
</> Bash  
  
pip install requests
```

Primer ejemplo con GET

```
import requests  
  
respuesta = requests.get("https://jsonplaceholder.typicode.com/users")  
  
print(respuesta.status_code)  
print(respuesta.text)
```

Convertir la respuesta a JSON

```
import requests  
  
respuesta = requests.get("https://jsonplaceholder.typicode.com/users")  
datos = respuesta.json()  
  
print(datos)  
print(type(datos))
```

Recorrer respuesta

```
import requests  
  
respuesta = requests.get("https://jsonplaceholder.typicode.com/users")  
usuarios = respuesta.json()  
  
for usuario in usuarios:  
    print(usuario["name"])
```

7. Enviar datos con POST

```
import requests

nuevo_usuario = {
    "name": "Pakova",
    "email": "pakova@mail.com"
}

respuesta = requests.post(
    "https://jsonplaceholder.typicode.com/users",
    json=nuevo_usuario
)

print(respuesta.status_code)
print(respuesta.json())
```

Diferencia entre **data** y **json**

- **data=** envía datos más crudos o formularios
- **json=** envía directamente JSON

Para APIs REST modernas, normalmente se usa **json=**.

8. Validaciones orientadas a QA

Cuando se prueba una API, no alcanza con mirar que “responda algo”.

Hay que validar:

- status code
- estructura del body
- campos obligatorios
- tipos de datos
- mensajes de error
- tiempos de respuesta
- comportamiento ante datos inválidos

Ejemplo de validaciones en un GET

Supongamos una respuesta:

```
<> JSON

{
  "id": 1,
  "nombre": "Ana",
  "email": "ana@mail.com"
}
```

Validaciones posibles

- status code = 200
- **id** existe
- **nombre** existe
- **email** existe
- **id** es numérico
- **nombre** es string
- **email** tiene formato esperado

9. Casos de prueba manuales para APIs

CP01 - Obtener lista de usuarios

Método: GET

Endpoint: **/usuarios**

Pasos:

1. Enviar request GET al endpoint.
2. Revisar status code.
3. Revisar body de respuesta.

Resultado esperado:

- status code 200
 - respuesta en formato JSON
 - devuelve lista de usuarios
-

CP02 - Obtener usuario existente

Método: GET

Endpoint: **/usuarios/1**

Resultado esperado:

- status code 200
 - devuelve datos del usuario solicitado
 - el campo **id** coincide con el solicitado
-

CP03 - Obtener usuario inexistente

Método: GET

Endpoint: **/usuarios/9999**

Resultado esperado:

- status code 404
 - mensaje de error claro o body vacío según definición
-

CP04 - Crear usuario válido

Método: POST

Endpoint: **/usuarios**

Body:

```
<> JSON  
  
{  
  "nombre": "Ana",  
  "email": "ana@mail.com",  
  "edad": 25  
}
```

Resultado esperado:

- status code 201
 - usuario creado correctamente
 - respuesta contiene identificador
-

CP05 - Crear usuario con campos faltantes

Resultado esperado:

- status code 400
 - mensaje indicando validación fallida
-

10. Casos borde y trampas tipo QA

Caso borde 1: campo faltante

Body enviado:

```
</> JSON

{
  "email": "ana@mail.com"
}
```

Qué validar:

- que no cree el usuario
 - que responda error controlado
 - que informe qué campo falta
-

Caso borde 2: tipo incorrecto

```
</> JSON

{
  "nombre": "Ana",
  "email": "ana@mail.com",
  "edad": "veinte"
}
```

Qué validar:

- rechazo por tipo inválido
 - status code correcto
 - mensaje claro
-

Caso borde 3: valor vacío

```
</> JSON

{
  "nombre": "",
  "email": "ana@mail.com",
  "edad": 20
}
```

Qué validar:

- validación de obligatorio
 - no alta del registro
-

Caso borde 4: usuario inexistente

GET /usuarios/99999

Qué validar:

- no devuelve 200 por error
 - responde 404 o el comportamiento documentado
-

Caso borde 5: body mal formado

Enviar JSON roto o inválido.

Qué validar:

- respuesta controlada
 - no error 500 innecesario
 - mensaje útil
-

Caso borde 6: datos duplicados

Crear dos veces el mismo usuario o email.

Qué validar:

- regla de unicidad
 - mensaje correcto
-

Caso borde 7: campos extra no esperados

```
</> JSON

{
  "nombre": "Ana",
  "email": "ana@mail.com",
  "edad": 20,
  "rol_admin": true
}
```

Qué validar:

- si el backend ignora o rechaza campos extra
 - que no cree permisos no autorizados
-

11. Automatización básica de APIs con Python

Ejemplo 1: validar status code

```
import requests

def test_obtener_usuarios():
    respuesta = requests.get("https://jsonplaceholder.typicode.com/users")
    assert respuesta.status_code == 200
```

Ejemplo 2: validar estructura

```
import requests

def test_usuario_tiene_campos_esperados():
    respuesta = requests.get("https://jsonplaceholder.typicode.com/users/1")
    data = respuesta.json()

    assert respuesta.status_code == 200
    assert "id" in data
    assert "name" in data
    assert "email" in data
```

Ejemplo 3: validar tipo de datos

```
import requests

def test_tipo_de_datos_usuario():
    respuesta = requests.get("https://jsonplaceholder.typicode.com/users/1")
    data = respuesta.json()

    assert isinstance(data["id"], int)
    assert isinstance(data["name"], str)
    assert isinstance(data["email"], str)
```

12. Automatización con pytest

Archivo sugerido:

```
test_api_usuarios.py
```

Contenido:

```
import requests

BASE_URL = "https://jsonplaceholder.typicode.com"

def test_get_usuarios_devuelve_200():
    respuesta = requests.get(f"{BASE_URL}/users")
    assert respuesta.status_code == 200

def test_get_usuarios_devuelve_lista():
    respuesta = requests.get(f"{BASE_URL}/users")
    data = respuesta.json()

    assert isinstance(data, list)
    assert len(data) > 0
```

```
def test_get_usuario_por_id():
    respuesta = requests.get(f"{BASE_URL}/users/1")
    data = respuesta.json()
```

```
    assert respuesta.status_code == 200
    assert data["id"] == 1
```

```
def test_usuario_contiene_campos_requeridos():
    respuesta = requests.get(f"{BASE_URL}/users/1")
    data = respuesta.json()
```

```
    assert "id" in data
    assert "name" in data
    assert "email" in data
```

```
def test_post_usuario():
```

```
    body = {
        "name": "Pakova",
        "email": "pakova@mail.com"
    }
```

```
    respuesta = requests.post(f"{BASE_URL}/users", json=body)
    data = respuesta.json()
```

```
    assert respuesta.status_code == 201
    assert data["name"] == "Pakova"
    assert data["email"] == "pakova@mail.com"
```

13. Buenas prácticas de testing de APIs

- separar URL base en una constante
- no repetir datos innecesariamente
- validar más que el status code
- usar nombres claros en los tests
- cubrir casos positivos y negativos
- no asumir que una respuesta correcta siempre tiene datos correctos
- revisar claves, valores y tipos

14. Ejercicio práctico del módulo

Consigna

Consumir una API de usuarios y validar:

1. que la respuesta sea 200
2. que el body sea una lista
3. que cada usuario tenga:
 - id
 - name
 - email
4. que **id** sea entero
5. que **name** y **email** sean texto

Ejemplo de solución

```
import requests

respuesta = requests.get("https://jsonplaceholder.typicode.com/users")
usuarios = respuesta.json()

if respuesta.status_code == 200 and isinstance(usuarios, list):
    for usuario in usuarios:
        if "id" in usuario and "name" in usuario and "email" in usuario:
            if isinstance(usuario["id"], int):
                print("Usuario válido:", usuario["name"])
            else:
                print("ID inválido:", usuario)
        else:
            print("Faltan campos:", usuario)
    else:
        print("La respuesta no es válida")
```

6 Módulo 6: Testing Automation con pytest

1. ¿Qué es pytest?

pytest es una herramienta para automatizar pruebas en Python.

Se usa para:

- validar lógica de negocio
 - testear APIs
 - detectar errores automáticamente
 - evitar regresiones
-

Instalación

```
</> Bash  
pip install pytest
```

Ejecutar tests

```
</> Bash  
pytest
```

Con detalle:

```
</> Bash  
pytest -v
```

2. Estructura básica de un test

Archivo

Los tests deben llamarse:

```
</> Bash  
test_*.py
```

Ejemplo simple

```
def test_suma():  
    assert 2 + 2 == 4
```

Concepto clave: **assert**

Sirve para verificar condiciones.

```
assert condicion
```

Si falla → el test falla ❌

Si pasa → el test pasa ✅

3. Buenas prácticas con asserts

❌ Incorrecto:

```
assert 2 + 2 == 5
```

✅ Correcto:

```
resultado = 2 + 2  
assert resultado == 4
```

Validar múltiples cosas

```
def test_usuario():  
    usuario = {"id": 1, "nombre": "Ana"}  
  
    assert "id" in usuario  
    assert usuario["id"] == 1
```

4. Organización de tests

Estructura recomendada

```
proyecto/
├── backend_usuarios.py
├── test_backend_usuarios.py
└── test_api.py
```

Separar por tipo

- lógica → **test_backend**
- APIs → **test_api**
- integración → **test_integracion**

5. Fixtures (🔥 MUY IMPORTANTE)

¿Qué es una fixture?

Es una función que prepara datos o contexto reutilizable.

Ejemplo básico

```
import pytest

@pytest.fixture
def usuario_base():
    return {
        "id": 1,
        "nombre": "Ana",
        "email": "ana@mail.com",
        "edad": 25
    }
```

Uso de fixture


```
@pytest.fixture
def lista_usuarios():
    return [
        {"id": 1, "nombre": "Ana"},
        {"id": 2, "nombre": "Luis"}
    ]
```

6. Parametrización de tests

Permite probar múltiples casos con menos código.

Ejemplo

```
import pytest

@pytest.mark.parametrize("a,b,resultado", [
    (2, 2, 4),
    (3, 5, 8),
    (10, 0, 10)
])
def test_suma(a, b, resultado):
    assert a + b == resultado
```

Ejemplo QA real

```
@pytest.mark.parametrize("edad,esperado", [
    (18, True),
    (17, False),
    (0, False),
    (-1, False)
])
def test_mayoria_edad(edad, esperado):
    assert (edad >= 18) == esperado
```

7. Manejo de excepciones en tests

Validar que un error ocurra

```
import pytest

def dividir(a, b):
    return a / b

def test_division_por_cero():
    with pytest.raises(ZeroDivisionError):
        dividir(10, 0)
```

8. Setup y Teardown

Se usan para preparar y limpiar contexto.

Ejemplo con fixture

```
@pytest.fixture
def archivo_temporal(tmp_path):
    archivo = tmp_path / "test.txt"
    archivo.write_text("hola")
    return archivo
```

Uso

```
def test_archivo(archivo_temporal):
    contenido = archivo_temporal.read_text()
    assert contenido == "hola"
```

9. Mocks (básico)

¿Qué es un mock?

Simula comportamientos externos (API, DB, etc.)

Ejemplo con API

```
from unittest.mock import patch
import requests

def obtener_datos():
    return requests.get("https://api.com").json()
```

Test con mock

```
def test_obtener_datos():
    with patch("requests.get") as mock_get:
        mock_get.return_value.json.return_value = {"ok": True}

        resultado = obtener_datos()

        assert resultado["ok"] is True
```

10. Testing de APIs con pytest

Ejemplo real

```
import requests

BASE_URL = "https://jsonplaceholder.typicode.com"

def test_get_users():
    r = requests.get(f"{BASE_URL}/users")
    assert r.status_code == 200

def test_user_structure():
    r = requests.get(f"{BASE_URL}/users/1")
    data = r.json()

    assert "id" in data
    assert "name" in data
    assert "email" in data
```

11. Casos borde en automatización

Caso 1: respuesta vacía

```
def test_lista_no_vacia():
    r = requests.get("https://jsonplaceholder.typicode.com/users")
    data = r.json()

    assert len(data) > 0
```

Caso 2: tipo incorrecto

```
def test_tipo_id():
    r = requests.get("https://jsonplaceholder.typicode.com/users/1")
    data = r.json()

    assert isinstance(data["id"], int)
```

Caso 3: campo inexistente

```
def test_campos():
    r = requests.get("https://jsonplaceholder.typicode.com/users/1")
    data = r.json()

    assert "email" in data
```

12. Reportes de pytest

Ejecutar con más detalle

```
<> Bash
pytest -v
```

Mostrar prints

```
<> Bash
pytest -s
```

Generar reporte simple

```
</> Bash  
pytest --tb=short
```

(Avanzado) HTML report

```
</> Bash  
pip install pytest-html  
pytest --html=reporte.html
```

13. Ejercicio práctico del módulo

Consigna

Crear tests automatizados para el sistema de usuarios del TP anterior:

Validar:

1. creación correcta
2. validaciones de errores
3. búsqueda
4. eliminación
5. duplicados

Ejemplo base

```
def test_crear_usuario():  
    usuarios = []  
    ok, mensaje = crear_usuario(usuarios, 1, "Ana", "ana@mail.com", 25)  
  
    assert ok is True  
    assert len(usuarios) == 1
```

14. Evaluación tipo QA Automation

Caso 1: input inválido

👉 validar que el test falle correctamente

Caso 2: datos corruptos

👉 validar que el sistema no rompa

Caso 3: comportamiento inesperado

👉 validar que el test detecte el bug

Caso 4: regresión

👉 cambiar código y verificar que tests fallen

15. Mentalidad QA Automation (clave)

Siempre preguntarte:

- ❌ ¿Qué pasa si esto cambia?
- ❌ ¿Qué pasa si la API falla?
- ❌ ¿Qué pasa si el dato es inválido?
- ❌ ¿Qué pasa si el sistema devuelve vacío?